

Demand Forecasting & Inventory Optimization Engine

Comprehensive Implementation Document

Project Code: DFIO-2026

Version: 1.0

Date: February 2026

Classification: Technical Architecture & Implementation Plan

Project Team

#	Name
1	Eman Ahmed Sayed Ali
2	Ahmed Arafa Hafez Abdelkhalek
3	Sama Ashraf Mahmoud Elezmazy
4	Hadeer Mahmoud Abdelsalam Ghoneim
5	Mark Mohsen Nasry
6	Fadi Milad Shahat

1. Project Overview

1.1 Problem Statement

Retail and supply chain organizations face persistent challenges in accurately predicting product demand across time horizons ranging from days to months. Inaccurate demand forecasts lead directly to two costly outcomes: **overstock** — excess capital tied in inventory, increased holding costs, and potential waste — and **stockout** — lost sales, customer dissatisfaction, and reputational damage. Traditional rule-based planning systems cannot adapt to complex patterns such as seasonality, promotions, supply disruptions, or sudden market shifts.

1.2 Business Objectives

#	Objective	Target
1	Reduce forecast error (MAPE)	< 10% on fast-moving products
2	Eliminate stockout events	Reduce by 30% within 3 months
3	Reduce excess inventory	Cut average overstock by 20%
4	Automate inventory planning	Replace 80% of manual planner effort
5	Provide real-time decision support	Dashboard refresh < 1 minute

1.3 Scope Definition

In Scope:

- Demand forecasting for multi-product, multi-warehouse retail dataset
- Statistical (SARIMA), probabilistic (Prophet), and gradient-boosting (LightGBM/XGBoost) models
- Ensemble forecasting combining complementary model strengths
- Inventory policy calculation: EOQ, Reorder Point, Safety Stock
- REST API for programmatic access to forecasts and inventory recommendations
- Containerized deployment on Azure with CI/CD pipeline
- MLOps pipeline with drift detection and automated retraining
- Interactive operations dashboard for business users

Out of Scope:

- Real-time point-of-sale integration (assumed batch ingestion)

- Pricing optimization or promotion planning
- Supplier management or procurement execution
- Deep Learning sequence models (LSTM, Transformer) — noted as future work

1.4 Assumptions & Constraints

Assumptions:

- Historical daily sales data is available for at least 2 years per product
- Products follow discernible seasonal or trend patterns
- Lead times and holding costs are known or estimable
- Deployment targets Azure cloud infrastructure

Constraints:

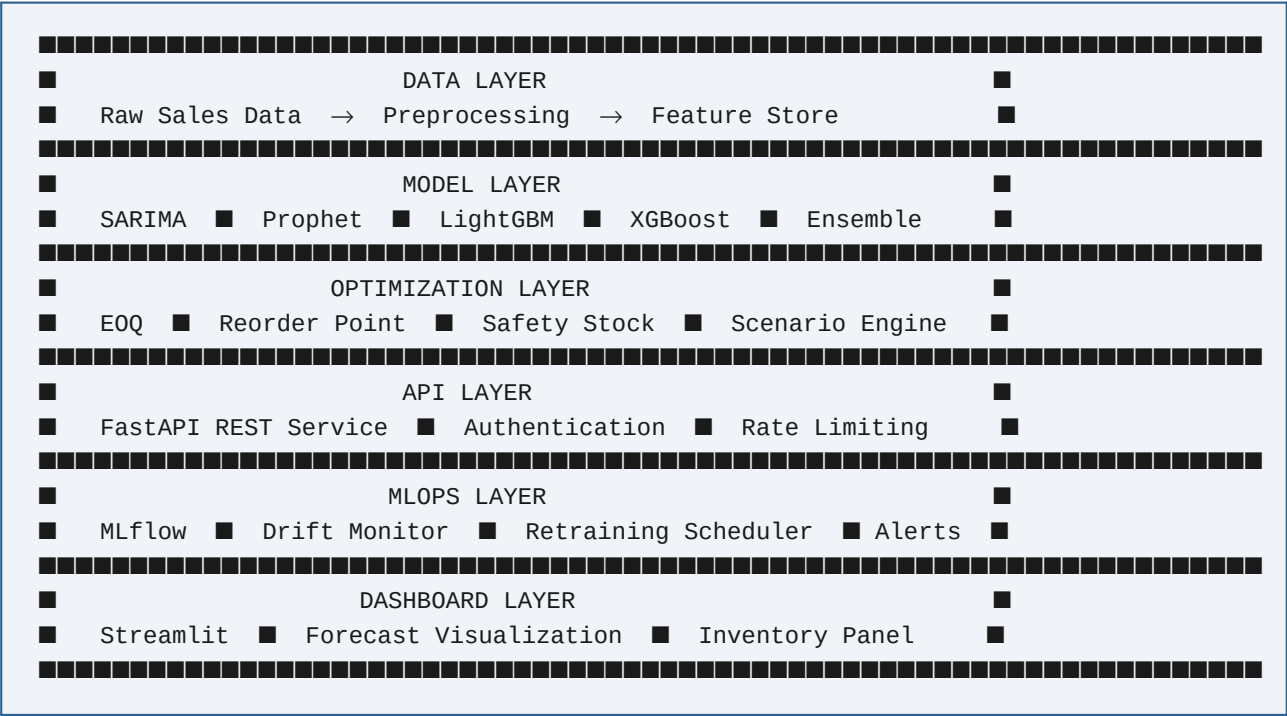
- Initial dataset is synthetic/simulated; real data ingestion is a subsequent phase
- Model retraining must complete within 2 hours for production feasibility
- API SLA: 200ms P95 response latency under 100 concurrent requests

1.5 Success Criteria

- **SC-1:** Forecast MAPE $\leq 10\%$ on held-out test set across $\geq 80\%$ of product SKUs
- **SC-2:** Inventory recommendations reduce simulated holding cost by $\geq 15\%$ vs. current policy
- **SC-3:** FastAPI service achieves $< 200\text{ms}$ P95 latency in load testing
- **SC-4:** Dashboard renders full forecast + inventory view in < 3 seconds
- **SC-5:** Automated retraining pipeline triggers correctly when MAPE drift exceeds 5%
- **SC-6:** All code, notebooks, and documentation are version-controlled and reproducible

2. System Architecture

2.1 High-Level Architecture



2.2 Data Flow

- 1. Ingestion:** Historical daily sales records are loaded from CSV or blob storage into the preprocessing pipeline.
- 2. Preprocessing:** Data is cleaned (outlier detection, missing value imputation), and features are engineered (lags, rolling statistics, calendar features, promotion flags).
- 3. Model Training:** Each model is trained on historical data with walk-forward cross-validation. Best parameters are selected and logged to MLflow.
- 4. Inference:** The API receives a product ID and horizon, queries the trained ensemble, and returns point forecasts with confidence intervals.
- 5. Optimization:** Forecast outputs feed the inventory optimization module, returning EOQ, reorder point, and safety stock recommendations.
- 6. Monitoring:** Production predictions are compared against actuals on a rolling basis. MAPE drift triggers automated retraining.
- 7. Presentation:** The dashboard polls the API and renders interactive charts and recommendation tables.

2.3 Layer Descriptions

Layer	Responsibility	Key Technologies
Data Layer	Ingestion, cleaning, feature engineering	Pandas, NumPy, Scikit-learn
Model Layer	Statistical & ML forecasting	Statsmodels, Prophet, LightGBM, XGBoost
Optimization Layer	EOQ, ROP, Safety Stock	SciPy, custom Python modules
API Layer	REST endpoints, validation, auth	FastAPI, Pydantic, Uvicorn
MLOps Layer	Tracking, drift detection, retraining	MLflow, Evidently, Azure Functions
Dashboard Layer	Interactive BI for operations teams	Streamlit, Plotly

3. Milestone Breakdown

Milestone 1 — Data Collection & Exploratory Data Analysis

Objective: Establish a clean, feature-rich, forecast-ready dataset and thoroughly understand its statistical properties before any modeling takes place.

Tasks

- Design synthetic dataset schema (products, stores, dates, quantities, prices, promotions)
- Generate 3 years of realistic daily demand data for 20 product SKUs across 5 store locations
- Inject controlled patterns: weekly/annual seasonality, promotional lift, trend, and Gaussian noise
- Introduce realistic data quality issues (1–3% missing values, occasional anomaly spikes)
- Perform missing value analysis and select imputation strategy
- Conduct outlier detection using IQR and Z-score methods
- Visualize time-series decomposition (trend, seasonal, residual) using STL decomposition
- Generate ACF and PACF plots to guide SARIMA parameter selection
- Document preprocessing decisions and EDA findings in a structured report

Tools & Technologies

Tool	Purpose
Python 3.11	Core language
Pandas / NumPy	Data manipulation & generation
Matplotlib / Seaborn	Static visualizations
Plotly	Interactive EDA charts
Statsmodels	STL decomposition, ACF/PACF
Jupyter Notebook	EDA environment

Deliverables

- data/demand_data.csv — 3-year multi-product daily dataset
- notebooks/01_EDA.ipynb — Complete EDA notebook
- src/preprocessing.py — Reusable preprocessing module
- reports/EDA_Report.html — Exported HTML report

Risks & Mitigation

Risk	Mitigation
Synthetic data may not represent real distribution	Validate statistical properties against published retail benchmarks
Over-cleaning removes genuine demand spikes	Use domain-informed thresholds; flag rather than delete outliers

Milestone 2 — Forecast Model Development

Objective: Train, evaluate, and select the best-performing combination of statistical and machine learning models, then derive inventory policy recommendations.

Tasks

- Split data into train (70%), validation (15%), and test (15%) using time-ordered splits
- Implement SARIMA with automatic (p,d,q)(P,D,Q) selection via AIC grid search
- Implement Facebook Prophet with custom seasonality, holiday effects, and uncertainty intervals
- Engineer lag features (1, 7, 14, 28 days), rolling mean/std, and calendar features
- Train LightGBM and XGBoost models with TimeSeriesSplit cross-validation
- Build weighted ensemble combining Prophet and LightGBM; optimize weights on validation set
- Evaluate all models on test set: MAPE, WAPE, SMAPE, RMSE, and Bias
- Implement EOQ, Reorder Point, and Safety Stock calculations
- Build a model comparison report with residual plots and error distributions

Tools & Technologies

Tool	Purpose
Statsmodels	SARIMA implementation
Prophet (Meta)	Prophet model
LightGBM	Gradient boosting
XGBoost	Gradient boosting
Optuna	Hyperparameter optimization
Scikit-learn	Cross-validation, metrics
SciPy	Statistical distributions for safety stock

Risks & Mitigation

Risk	Mitigation
SARIMA fails to converge for noisy series	Fall back to Prophet or ML model; set hard timeout on grid search
Overfitting on synthetic training data	Enforce strict walk-forward CV; monitor validation MAPE
Inventory formulas sensitive to lead time assumptions	Perform sensitivity analysis across lead time $\pm 20\%$

Milestone 3 — Azure Deployment & REST API

Objective: Package the trained forecasting system into a production-grade REST API, containerize it with Docker, and deploy to Azure.

API Endpoints

Method	Endpoint	Description
GET	/health	Service health check, version info, uptime
GET	/products	List all available product SKUs
POST	/forecast	Generate demand forecast for a product & horizon
POST	/inventory	Get EOQ, reorder point, safety stock
GET	/models/info	Return active model version metadata

Tools & Technologies

Tool	Purpose
FastAPI	REST API framework
Uvicorn	ASGI server
Pydantic v2	Request/response validation
Docker	Containerization
Azure Container Apps	Serverless deployment
GitHub Actions	CI/CD pipeline
Locust	Load testing

Risks & Mitigation

Risk	Mitigation
Cold start latency on Azure Container Apps	Configure minimum replicas = 1 to prevent cold starts
Model file too large for container	Store models in Azure Blob Storage; load lazily from URL
API key exposure	Use Azure Key Vault; never hardcode secrets

Milestone 4 — MLOps, Monitoring & Dashboard

Objective: Instrument the full ML lifecycle with experiment tracking, production monitoring, automated retraining, and deliver an interactive operations dashboard.

Dashboard Views

View	Content
Overview Page	KPI summary cards: MAPE, stockout rate, inventory turnover
Forecast Page	Product selector, historical vs. forecast chart, confidence intervals
Inventory Page	Recommendations table, stock status traffic lights, EOQ chart
Monitoring Page	Rolling MAPE over time, drift alerts, model version history

Tools & Technologies

Tool	Purpose
MLflow	Experiment tracking & model registry
Streamlit	Dashboard framework
Azure Functions	Scheduled monitoring & retraining
Plotly	Interactive dashboard charts
Evidently	Drift detection (PSI)
SMTP / Slack API	Alerting on drift or retraining events

Risks & Mitigation

Risk	Mitigation
False drift alerts from seasonal changes	Use seasonally-adjusted baselines for drift comparison
Retraining degrades performance	Gate promotion: only promote if new MAPE < current production MAPE
Dashboard slow to load large datasets	Pre-aggregate summaries server-side; cache in session state

Milestone 5 — Final Report & Presentation

Objective: Produce comprehensive project documentation suitable for portfolio, academic, and stakeholder submission, and prepare a compelling demo.

Deliverables

- docs/Project_Proposal.md — Problem, objectives, scope, and team
- docs/Project_Plan.md — Gantt chart, milestones, deliverables, resource allocation
- docs/Risk_Assessment.md — Risk register with probability-impact matrix
- docs/KPIs.md — KPI definitions with target values and measurement methods
- docs/Literature_Review.md — Survey of forecasting methodologies
- README.md — Master documentation with architecture diagram and setup guide
- Demo script / recording — End-to-end walkthrough for presentation

4. Data Design

4.1 Forecast-Ready Dataset Schema

Column	Type	Description
date	DATE	Daily timestamp (YYYY-MM-DD)
product_id	STRING	SKU identifier (e.g., PROD_001)
store_id	STRING	Store or warehouse location
demand	FLOAT	Units sold — primary target variable
price	FLOAT	Unit selling price
promotion_flag	INT (0/1)	Whether a promotion was active
holiday_flag	INT (0/1)	National or regional holiday indicator
day_of_week	INT (0–6)	Monday = 0
week_of_year	INT (1–52)	ISO week number
month	INT (1–12)	Calendar month
lag_1	FLOAT	Demand 1 day ago
lag_7	FLOAT	Demand 7 days ago
lag_14	FLOAT	Demand 14 days ago
lag_28	FLOAT	Demand 28 days ago
rolling_mean_7	FLOAT	7-day rolling average demand
rolling_std_7	FLOAT	7-day rolling standard deviation
rolling_mean_30	FLOAT	30-day rolling average demand
trend_index	FLOAT	Linear time index for trend capture

4.2 Feature Engineering Strategy

Lag Features: Capture autocorrelation by embedding past demand values as predictors. Lags 1, 7, 14, and 28 capture day-to-day, weekly, bi-weekly, and monthly carry-over effects.

Rolling Statistics: Smooth short-term noise by computing 7-day and 30-day rolling means and standard deviations. The rolling mean captures local trend; the rolling std captures

volatility.

Calendar Features: Day-of-week, week-of-year, and month encode cyclical seasonality, optionally encoded as sin/cos transformations to preserve circularity.

Event Features: Binary flags for promotions and public holidays capture demand shocks not explainable from historical demand alone.

Trend Index: A monotonically increasing integer representing elapsed time, allowing models to capture long-term growth or decline.

4.3 Time-Series Cross-Validation

Walk-forward (expanding window) cross-validation is used, which respects temporal ordering and prevents look-ahead bias. Each fold uses all data up to a cutoff point for training and the subsequent N days as the validation set. Folds are generated at monthly intervals over the last 12 months of training data, producing 12 validation windows.

4.4 Evaluation Metrics

Metric	Formula	Interpretation
MAPE	$\text{mean}(\text{actual} - \text{forecast} / \text{actual}) \times 100$	Percentage error; scale-independent; sensitive to zero actuals
WAPE	$\text{sum}(\text{actual} - \text{forecast}) / \text{sum}(\text{actual}) \times 100$	Volume-weighted; robust to low-demand periods
SMAPE	$\text{mean}(2 \times \text{actual} - \text{forecast} / (\text{actual} + \text{forecast})) \times 100$	Symmetric; avoids asymmetric penalty of MAPE
RMSE	$\sqrt{\text{mean}((\text{actual} - \text{forecast})^2)}$	Penalizes large errors heavily; same unit as demand
Bias	$\text{mean}(\text{forecast} - \text{actual})$	Signed; reveals systematic over- or under-forecasting

5. Inventory Optimization Logic

5.1 Economic Order Quantity (EOQ)

The EOQ minimizes total inventory cost by balancing ordering costs against holding costs.

$$EOQ = \sqrt{[(2 \times D \times S) / H]}$$

Variable	Definition
D	Annual demand in units — derived from the 365-day forecast
S	Fixed ordering cost per purchase order (currency)
H	Annual holding cost per unit (currency)

5.2 Reorder Point (ROP)

The Reorder Point defines the inventory level at which a new order must be placed.

$$ROP = (Average\ Daily\ Demand \times Lead\ Time) + Safety\ Stock$$

5.3 Safety Stock

Safety stock absorbs uncertainty in both demand and lead time.

$$Safety\ Stock = Z \times \sigma_d \times \sqrt{(Lead\ Time)}$$

Variable	Definition
Z	Service level z-score (e.g., 1.65 for 95%, 2.05 for 98%)
σ_d	Standard deviation of daily demand from forecast confidence intervals
Lead Time	Supplier delivery time in days

5.4 Service Level Definition

The probability of not experiencing a stockout during any given replenishment cycle. A 95% service level means stockouts are acceptable in only 5% of replenishment cycles.

5.5 Scenario Testing

Scenario	Lead Time	Service Level	Purpose
Optimistic	–20% of base	90%	Cost minimization
Base	Configured value	95%	Standard operating policy
Conservative	+20% of base	98%	Risk-averse / disruption-resilient

6. Deployment & DevOps Plan

6.1 Containerization Strategy

The application is packaged as a Docker image following a **multi-stage build** pattern:

- **Builder Stage:** Full Python environment with development dependencies for compiling packages
- **Production Stage:** Minimal runtime environment with only production dependencies, reducing image size by ~60%

The container image is stored in Azure Container Registry (ACR) and tagged with the Git commit SHA and semantic version for full traceability.

6.2 Environment Configuration

Variable	Purpose
API_SECRET_KEY	API authentication key — stored in Azure Key Vault
MODEL_STORAGE_URL	Azure Blob URL for model artifacts
MLFLOW_TRACKING_URI	MLflow server endpoint
LOG_LEVEL	Logging verbosity (INFO / DEBUG / WARNING)
MAX_FORECAST_HORIZON	Maximum allowed forecast days per request

6.3 CI/CD Pipeline

Stage	Action
Pull Request	Lint (flake8, black), Unit Tests (pytest), Integration Tests
Merge to main	Build Docker image, push to Azure Container Registry
Staging deployment	Deploy to staging, run smoke tests
Production promotion	Manual approval gate, then deploy to production

6.4 Logging & Monitoring Plan

Structured JSON logging is implemented at all layers. Each log entry includes: timestamp, level, service, endpoint, request_id, duration_ms, and error (if applicable). Logs are collected by Azure Monitor and queried via Log Analytics. Alerting rules notify on:

- Error rate > 1% over a 5-minute window

- P95 latency > 200ms over a 5-minute window
- Model MAPE drift > 5% over a 7-day rolling window

7. MLOps Strategy

7.1 Model Versioning

Every training run is logged to MLflow with parameters, metrics, and serialized model artifacts. Models progress through three stages in the Model Registry:

Stage	Meaning
None	Freshly trained, under evaluation
Staging	Passed validation thresholds, pending A/B testing
Production	Serving live traffic

7.2 Drift Detection

Data Drift (Covariate Shift): Input feature distributions are compared between training baseline and current production data using Population Stability Index (PSI). $PSI > 0.25$ indicates significant drift.

Concept Drift (Performance Degradation): Rolling 7-day MAPE is computed on actuals vs. production predictions. If $MAPE > training_MAPE \times 1.05$ (5% relative degradation), retraining is triggered.

7.3 Retraining Triggers

Trigger Type	Condition	Action
Scheduled	Weekly, every Monday 02:00 UTC	Full retrain on sliding 2-year window
Drift-based	MAPE relative increase > 5%	Immediate retrain + alert
Data volume	New data exceeds 10,000 accumulated rows	Retrain and re-evaluate
Manual	On-demand via API endpoint	Immediate retrain

7.4 Monitoring Metrics

Metric	Healthy Range	Action if Violated
Production MAPE	< 12%	Trigger drift investigation
Prediction latency P95	< 200ms	Scale out API replicas
Data pipeline freshness	< 24 hours	Alert + investigate source
Feature PSI	< 0.25	Flag for team review
Model registry promotions	≥ 1/month	Healthy retraining cadence

8. KPIs & Success Metrics

8.1 Forecast Accuracy KPIs

KPI	Definition	Target	Measurement
MAPE	Mean Absolute Percentage Error	< 10%	Weekly on held-out actuals
WAPE	Weighted Absolute Percentage Error	< 8%	Weekly on held-out actuals
Bias	Mean signed forecast error	±2% of mean demand	Weekly
Forecast Coverage	% SKUs with MAPE < 15%	≥ 85%	Monthly

8.2 Inventory KPIs

KPI	Definition	Target	Measurement
Stockout Rate	% of days with zero stock	< 2%	Daily
Inventory Turnover	Annual demand / avg inventory	≥ 8×	Monthly
Overstock Ratio	On-hand / ROP	< 1.5	Monthly
Fill Rate	% orders fulfilled from stock	≥ 98%	Weekly
Holding Cost Savings	% reduction vs. baseline	≥ 15%	Quarterly

8.3 System Performance KPIs

KPI	Definition	Target	Measurement
API Latency P95	95th percentile response time	< 200ms	Real-time
API Uptime	% of time service is reachable	≥ 99.9%	Continuous
Dashboard Load Time	Time to full page render	< 3 seconds	Synthetic monitoring
Retraining Success Rate	% of triggered retrains that complete	≥ 95%	Per trigger event

9. Risk Assessment

9.1 Technical Risks

Risk	Prob.	Impact	Mitigation
Model overfitting to synthetic data	Medium	High	Strict walk-forward CV; regularization; validate on OOD scenarios
SARIMA convergence failure	Medium	Medium	Fallback to Prophet; set iteration timeout
API memory exhaustion	Low	High	Load models at startup; lazy loading for unused SKUs
Docker build failure in CI	Low	Medium	Pin all dependency versions; add retry logic

9.2 Data Risks

Risk	Prob.	Impact	Mitigation
Insufficient data for new SKUs	High	High	Cold-start: use category-average model; require 30+ data points minimum
Data quality issues causing failure	Medium	High	Pre-training data validation checks; reject corrupted batches
Feature leakage in lag engineering	Medium	High	Enforce temporal ordering strictly; use time-aware splits
Low-demand period noise	Medium	Low	Use WAPE metric to de-emphasize low-demand SKUs

9.3 Deployment Risks

Risk	Prob.	Impact	Mitigation
Azure service outage	Low	High	Configure multi-region fallback; implement circuit breaker
API key exposure in logs	Medium	High	Mask sensitive fields; audit log configuration
Deployment rollback failure	Low	High	Maintain previous Docker image tag; document rollback procedure
Unexpected cost from auto-scaling	Medium	Medium	Set maximum replica limits; configure cost alerts in Azure

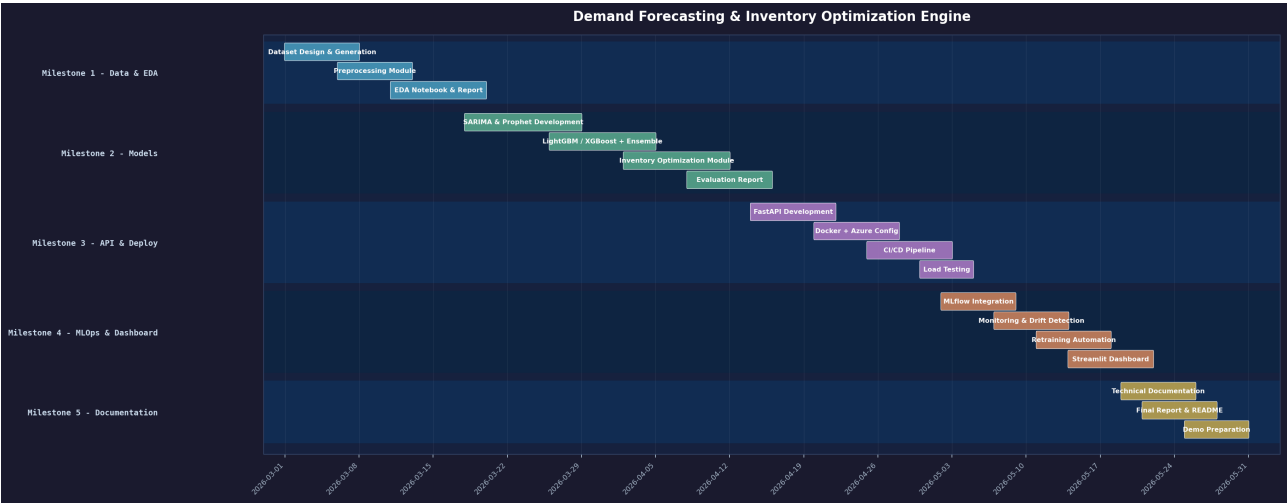
10. Timeline & Gantt Description

10.1 16-Week Execution Plan

Weeks	Phase	Key Activities	Miles tone
1–3	Data Foundation	Dataset generation, preprocessing pipeline, full EDA notebook	M1
4–7	Model Development	SARIMA, Prophet, LightGBM, ensemble, inventory logic, evaluation report	M2
8–10	API & Deployment	FastAPI, Docker, Azure Container Apps, CI/CD, load testing	M3
11–14	MLOps & Dashboard	MLflow tracking, drift monitoring, retraining automation, Streamlit	M4
15–16	Docs & Demo	All documentation, README, final presentation and demo recording	M5

10.2 Project Gantt Chart

The chart below illustrates the detailed task breakdown across all five milestones, color-coded by phase, spanning from March 2026 through May 2026.



11. Conclusion

11.1 Expected Business Impact

The Demand Forecasting & Inventory Optimization Engine is designed to deliver measurable operational and financial improvements. By achieving forecast MAPE below 10%, organizations can make inventory decisions grounded in data rather than intuition. The inventory optimization layer translates forecasts into concrete policy recommendations — reducing holding costs by an estimated 15–20% and cutting stockout events by 30%, representing a significant return on the development investment.

The REST API architecture democratizes access to forecasting capabilities, allowing integration with ERP systems, procurement tools, and custom business applications without requiring data science expertise at the point of consumption. The Streamlit dashboard further extends access to non-technical planners and operations managers.

11.2 Scalability Considerations

- The API layer scales horizontally via Azure Container Apps auto-scaling based on request queue depth
- The model layer supports adding new SKUs without architectural changes
- MLflow's model registry supports hundreds of concurrent model versions with no performance degradation
- For catalogs > 10,000 SKUs, migrate to Azure ML Online Endpoints with async job queues

11.3 Future Improvements

Area	Improvement
Deep Learning	Implement Temporal Fusion Transformer (TFT) for global multi-SKU modeling
Causal AI	Incorporate external signals: macro indicators, weather, competitor pricing
Real-time Streaming	Replace batch ingestion with Azure Event Hub for near-real-time forecasting
Explainability	Add SHAP values to API responses for interpretable forecast drivers
A/B Testing	Implement shadow-mode testing before fully promoting new model versions
Multi-echelon Inventory	Extend optimization to multi-warehouse, multi-distributor supply networks

This document is intended as the authoritative technical specification and implementation guide for the Demand Forecasting & Inventory Optimization Engine. It should be maintained alongside the codebase and updated as the system evolves.

Document prepared by: Project Team **Review status:** Ready for submission **Next review:** End of Milestone 1